LeafGuard

Plant Disease Detection Platform

Software Architecture Project Report

Vighnesh PM

ID: 2025H1120159P

Under the guidance of

Dr. Tanmay Mahapatra

BITS Pilani, Pilani Campus

April 2026

Contents

1 Executive Summary and Problem Context **4**
- 1.1 Project Overview . . . 4
- 1.2 Problem Statement . . . 4
- 1.3 Key Contributions . . . 4
- 1.4 Document Structure . . . 4

2 Stakeholders, Concerns, and Constraints **5**
- 2.1 Architecture Business Cycle . . . 5
- 2.2 Stakeholder Identification . . . 5
- 2.3 Stakeholder Concerns Mapping . . . 5
- 2.4 Architectural Constraints . . . 6
  - 2.4.1 Technical Constraints . . . 6
  - 2.4.2 Business Constraints . . . 6
  - 2.4.3 Organizational Constraints . . . 6

3 Functional Requirements **7**
- 3.1 Requirements Overview . . . 7
- 3.2 Functional Requirements Specification . . . 7
- 3.3 Use Case: Primary Prediction Flow . . . 8
- 3.4 Use Case Narrative: Predict Disease . . . 8

4 Quality Attributes and Scenarios **9**
- 4.1 Quality Attribute Overview . . . 9
- 4.2 Utility Tree . . . 9
- 4.3 Quality Attribute Scenarios . . . 10
  - 4.3.1 QAS-1: Performance . . . 10
  - 4.3.2 QAS-2: Modifiability . . . 10
  - 4.3.3 QAS-3: Testability . . . 10
  - 4.3.4 QAS-4: Reliability . . . 11
  - 4.3.5 QAS-5: Availability . . . 11
  - 4.3.6 QAS-6: Security . . . 11
  - 4.3.7 QAS-7: Usability . . . 11
  - 4.3.8 QAS-8: Interoperability . . . 12
  - 4.3.9 QAS-9: Deployability . . . 12
  - 4.3.10 QAS-10: Explainability . . . 12

5 Architecture Background **13**
- 5.1 Design Rationale . . . 13
- 5.2 Analysis of Results . . . 13
- 5.3 Assumptions . . . 13

6 Architecture Overview and Styles **15**
6.1 Architectural Approach . . . 15
6.2 Layered Architecture (System-Level) . . . 15
6.2.1 Layer Definition . . . 15
6.2.2 Layer Responsibilities . . . 15
6.2.3 Rationale and Tradeoffs . . . 15
6.3 Pipe-and-Filter (ML Inference Subsystem) . . . 16
6.3.1 Pattern Description . . . 16
6.3.2 Filter Chain . . . 16
6.3.3 Rationale and Tradeoffs . . . 16
6.4 Client-Server Pattern . . . 16

7 Architecture Views and Diagrams **17**
7.1 View 1: System Context (C4 Level 1) . . . 17
7.1.1 Primary Presentation . . . 17
7.1.2 Element Catalog . . . 17
7.2 View 2: Container View (C4 Level 2) . . . 17
7.2.1 Primary Presentation . . . 18
7.2.2 Element Catalog . . . 18
7.2.3 Container Interfaces . . . 18
7.3 View 3: Backend Module View . . . 18
7.3.1 Primary Presentation . . . 19
7.3.2 Dependency Rules . . . 19
7.4 View 4: Pipe-and-Filter Runtime View (C&C) . . . 20
7.4.1 Primary Presentation . . . 20
7.4.2 Filter Behavior Catalog . . . 20
7.5 View 5: Sequence Diagram – Predict Request . . . 21
7.5.1 Successful Prediction Flow . . . 21
7.5.2 Error Flow . . . 21
7.6 View 6: Deployment View . . . 22
7.7 View 7: UI Interaction State Diagram . . . 22
7.8 Variability Guide . . . 22

8 Attribute-Driven Design (ADD) Walkthrough **23**
8.1 Design Drivers . . . 23
8.2 Iteration 1: System Decomposition . . . 23
8.3 Iteration 2: Inference Pipeline Decomposition . . . 23
8.4 Iteration 3: Species/Disease Modeling Strategy . . . 24
8.5 Iteration 4: UX and Interaction Design . . . 24
8.6 ADD Decomposition Tree . . . 24

9 ML Subsystem Architecture Deep Dive **25**
9.1 Overview . . . 25
9.2 Inference Data Path . . . 25

9.3 Model Catalog . . . 25
9.3.1 Species Classifier (CNN) . . . 25
9.3.2 AS Disease Model (Hybrid CNN + SVM) . . . 26
9.3.3 PC Disease Model (Hybrid CNN + SVM) . . . 26
9.4 Model Routing Architecture . . . 26
9.5 Demo Mode Heuristic . . . 26

10 Tradeoffs, Tactics, and Sensitivity Analysis **28**
10.1 Architectural Tactics . . . 28
10.2 ATAM-Lite: Sensitivity Points . . . 28

11 Verification, Testing, and Evolution Plan **29**
11.1 Test Strategy . . . 29
11.2 Current Test Coverage . . . 29
11.3 ADD Validation Evidence . . . 29
11.4 Architectural Decision Records (ADR Summary) . . . 30
11.5 Evolution Roadmap . . . 30

12 Glossary of Terms **31**

13 Other Information **32**
13.1 Technology Stack Summary . . . 32
13.2 Repository Structure . . . 32
13.3 Build and Run Instructions . . . 32

References **34**

1 Executive Summary and Problem Context

1.1 Project Overview

LeafGuard is an AI-powered plant disease detection platform designed for scientific research on two arid-zone tree species: **Acacia Senegal (AS)** and **Prosopis Cineraria (PC)**. The system accepts leaf images paired with morphological metadata and performs a two-stage classification: first identifying the species, then predicting whether the leaf is **Healthy** or **Unhealthy**.

By combining Convolutional Neural Networks (CNNs) with Support Vector Machines (SVMs) in a hybrid architecture, LeafGuard achieves high classification accuracy while maintaining interpretability through confidence scores and structured result summaries.

1.2 Problem Statement

Manual identification of plant diseases in arid-zone species suffers from:

- **Subjectivity:** Visual assessment varies between researchers and across environmental conditions.
- **Time Cost:** Manual examination of thousands of leaf samples is prohibitively slow.
- **Expertise Requirement:** Accurate diagnosis requires trained botanists with species-specific knowledge.
- **Reproducibility:** Manual assessments lack consistency required for longitudinal studies.

1.3 Key Contributions

1. **Hybrid ML Architecture:** CNN image feature extraction with SVM classification, augmented by tabular metadata, achieving 96–99% accuracy.
2. **Species-Specific Disease Models:** Separate disease classifiers for AS and PC accounting for morphological differences.
3. **Modular Pipeline Design:** Pipe-and-filter architecture enabling independent evolution and testing of each inference stage.
4. **Research-Grade Web Interface:** A glassmorphism-styled dashboard providing confidence visualizations and structured result interpretation.

1.4 Document Structure

This report follows the conventions of *Software Architecture in Practice* (4th Edition) by Bass, Clements, and Kazman [1]. It covers stakeholder analysis, functional requirements, quality attributes, architectural views, design decisions, ML subsystem details, tradeoff analysis, and verification strategy.

2 Stakeholders, Concerns, and Constraints

2.1 Architecture Business Cycle

Following the ABC framework from Bass et al. [1], LeafGuard's architecture is shaped by the interplay between stakeholders, technical constraints, quality requirements, and the development organization.

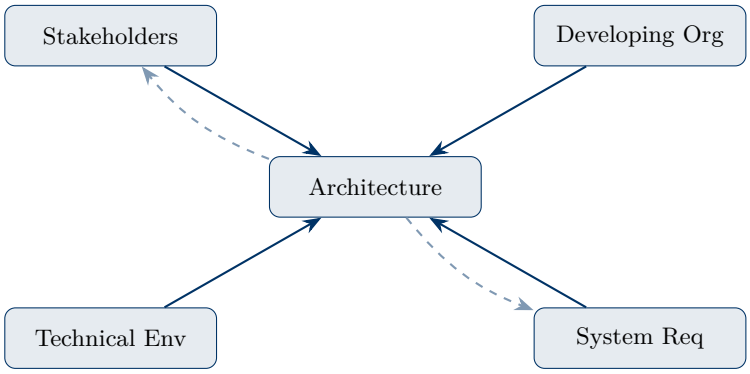

Figure 2.1: Architecture Business Cycle (adapted from Bass et al. [1])

2.2 Stakeholder Identification

Table 2.1: Stakeholder Registry

Stakeholder	Concerns	Priority
Research User	Accurate predictions, interpretable results, usable interface	High
Lab Supervisor	Reproducible results, audit trail, result confidence	High
ML Engineer	Model accuracy, inference latency, pipeline modularity	High
Frontend Developer	Clear API contracts, responsive UI, error handling	Medium
Backend Developer	Maintainable codebase, testable components	Medium
System Administrator	Deployment simplicity, monitoring, resource usage	Medium
Domain Expert	Species-specific accuracy, metadata relevance	High

2.3 Stakeholder Concerns Mapping

Table 2.2: Stakeholder-to-Quality Attribute Mapping

Concern	Quality Attributes Impacted
Prediction accuracy	Performance, Reliability
Result interpretability	Explainability, Usability
System responsiveness	Performance
Easy model updates	Modifiability
Independent stage testing	Testability
Secure input handling	Security
Quick deployment	Deployability
Stable API contracts	Interoperability

2.4 Architectural Constraints

2.4.1 Technical Constraints

1. **Python ML Ecosystem:** Models trained using TensorFlow/Keras and scikit-learn require a Python-based backend.
2. **Pre-trained Model Artifacts:** Three `.h5` CNN models and corresponding SVM `.pkl` files must be loaded at startup.
3. **Image Format:** Input images must be converted to 224×224 grayscale for CNN inference.
4. **Metadata Schema:** Exactly 10 tabular features in a fixed order must match the SVM training schema.

2.4.2 Business Constraints

1. **Research Context:** Developed within an academic research lab requiring rapid iteration.
2. **Two-Species Scope:** Current models support only AS and PC; architecture must accommodate future species.
3. **Local-First Deployment:** Initial deployment targets local machines with optional cloud migration.
4. **Demo Mode:** The system must function without model files for development purposes.

2.4.3 Organizational Constraints

1. **Small Team:** Favoring simplicity and maintainability over complex distributed architectures.
2. **Student/Research Codebase:** Must be readable and well-documented for knowledge transfer.

3 Functional Requirements

3.1 Requirements Overview

This chapter presents the functional requirements for LeafGuard, following traceability conventions recommended in Bass et al. [1].

3.2 Functional Requirements Specification

Table 3.1: Functional Requirements

FR ID	Requirement Description
FR-01	System shall accept a user-uploaded leaf image via the web interface.
FR-02	System shall validate image media type (JPEG, PNG, WebP) before backend inference.
FR-03	System shall enforce maximum image size of 10 MB at API boundary.
FR-04	System shall collect 9 user-facing metadata fields plus 1 derived field (Damage_missing) for inference.
FR-05	System shall validate Week as integer in range 1 to 52.
FR-06	System shall support Yes/No metadata controls in frontend toggle pill UI.
FR-07	System shall classify species as Acacia Senegal (AS) or Prosopis Cineraria (PC).
FR-08	System shall classify health status as Healthy or Unhealthy.
FR-09	System shall include species confidence score in API response.
FR-10	System shall include health confidence score in API response.
FR-11	System shall return a human-readable summary interpretation.
FR-12	System shall expose a health endpoint for service status checks.
FR-13	System shall expose pipeline stage metadata endpoint.
FR-14	System shall support demo heuristic inference when models are unavailable.
FR-15	System shall auto-dismiss error banner after configured timeout (4 s).
FR-16	System shall display results in dashboard format with confidence visualization.
FR-17	System shall support “Analyze Another Leaf” reset flow.

3.3 Use Case: Primary Prediction Flow

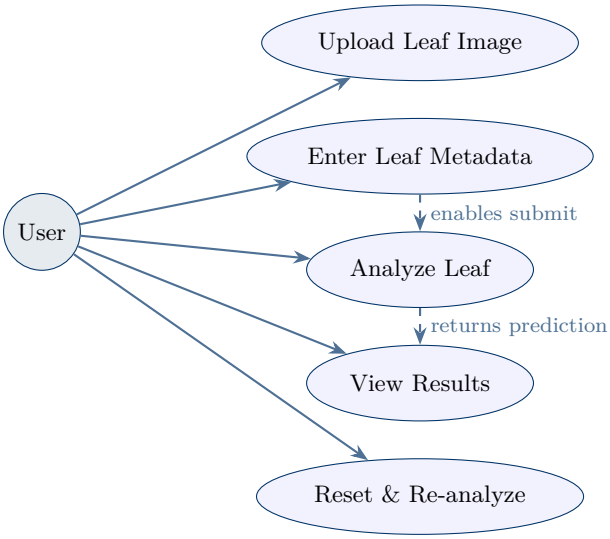

Figure 3.1: Use Case Diagram: LeafGuard Primary Prediction Flow

3.4 Use Case Narrative: Predict Disease

Table 3.2: Use Case: Predict Leaf Disease

Field	Description
Name	Predict Leaf Disease
Actor	Research User
Precondition	User has a leaf image and knows metadata values
Trigger	User clicks "Analyze Leaf" button
Main Flow	1. User uploads leaf image via drag-and-drop or file browser
	2. User fills 9 metadata fields using toggle pills and week input
	3. User clicks "Analyze Leaf"
	4. Frontend sends multipart POST to `/api/predict`
	5. Backend validates image type, size, and metadata
	6. Pipeline preprocesses image to 224×224 grayscale
	7. Pipeline validates and encodes metadata to 10-feature vector
	8. Species classifier predicts AS or PC with confidence
	9. Disease classifier routes to species-specific model
	10. Result formatter assembles structured JSON response
	11. Dashboard displays species, health, confidence scores, and summary
Postcondition	User sees prediction dashboard with species, health status, and confidence
Alternative	5a. Invalid type/size → HTTP 400 error
	5b. Invalid metadata → HTTP 422 error
	6–10. Models unavailable → demo heuristic inference

4 Quality Attributes and Scenarios

4.1 Quality Attribute Overview

Following Bass et al. [1], quality attributes are specified as measurable scenarios with six parts: *source*, *stimulus*, *environment*, *artifact*, *response*, and *response measure*.

4.2 Utility Tree

The utility tree prioritizes quality attributes by business importance (H/M/L) and technical risk (H/M/L), following the ATAM methodology [5].

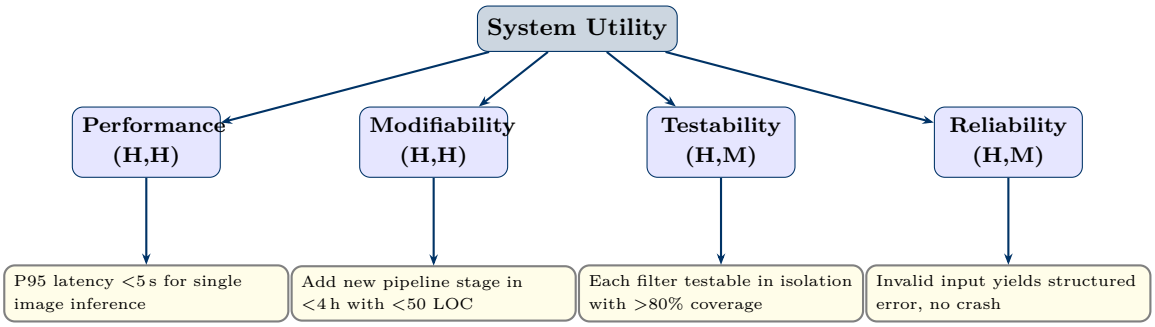

Figure 4.1: Utility Tree – Top Priority Quality Attributes

Table 4.1: Complete Utility Tree Leaf Nodes

QA	Refinement	(I,R)	Scenario
Performance	Inference latency	(H,H)	P95 prediction within 5 seconds under normal load
Performance	Model loading	(H,M)	Models preloaded at startup; no per-request overhead
Modifiability	Pipeline extension	(H,H)	New filter added in <4 hours; existing tests pass
Modifiability	Model replacement	(H,M)	Replace model without code changes beyond config
Testability	Stage isolation	(H,M)	Each filter testable with mock data dictionary input
Testability	Integration coverage	(M,M)	Full pipeline demo-mode test passes without model files
Reliability	Input validation	(H,M)	Malformed metadata returns HTTP 422 with message
Reliability	Graceful degradation	(M,H)	Falls back to demo heuristic when models unavailable
Availability	Health monitoring	(M,L)	`/api/health` returns 200 within 100 ms
Security	Input sanitization	(H,M)	Non-image uploads rejected at route layer
Usability	Error feedback	(M,M)	Error banner auto-dismissed after 4 seconds
Usability	Result clarity	(H,M)	Dashboard shows species, health, confidence, summary
Interop.	API stability	(M,L)	Multipart API contract unchanged across updates
Deployability	Env portability	(M,M)	Deploy to cloud with only env variable changes
Explainability	Result interpretation	(H,M)	Summary explains prediction in natural language

4.3 Quality Attribute Scenarios

4.3.1 QAS-1: Performance

Table 4.2: QAS-1: Performance

Element	Description
Source	Research user via web browser
Stimulus	Submits one leaf image (<10 MB) with 9 metadata fields
Environment	Normal operation, single-user, models preloaded
Artifact	ML inference pipeline (6 filters) and API route layer
Response	Returns structured JSON with species, health, and confidence
Response Measure	P95 end-to-end $\leq$ 5 s; preprocessing $\leq$ 200 ms; classification $\leq$ 2 s

4.3.2 QAS-2: Modifiability

Table 4.3: QAS-2: Modifiability

Element	Description
Source	ML engineer or developer
Stimulus	Request to add a "severity score" inference stage
Environment	Design time; development environment
Artifact	Pipeline module (`pipeline/` directory)
Response	New `BaseFilter` subclass created; registered in factory
Response Measure	Completed in <4 hours; <50 new LOC; all existing tests pass

4.3.3 QAS-3: Testability

Table 4.4: QAS-3: Testability

Element	Description
Source	Developer or CI system
Stimulus	Run test suite to verify each processing stage independently
Environment	Demo mode (no model files)
Artifact	Individual pipeline filters and utility functions
Response	Each filter produces expected output keys given controlled input
Response Measure	$\geq$80% coverage; all 8+ unit tests pass; integration test verifies full pipeline

4.3.4 QAS-4: Reliability

Table 4.5: QAS-4: Reliability

Element	Description
Source	External user or automated client
Stimulus	Submits invalid input: malformed JSON, non-image file, or out-of-range Week
Environment	Normal operation
Artifact	API route layer and metadata validation
Response	Returns HTTP 400/422 with descriptive message; no crash
Response Measure	100% invalid inputs produce structured errors; zero unhandled 500s

4.3.5 QAS-5: Availability

Table 4.6: QAS-5: Availability

Element	Description
Source	Monitoring system or load balancer
Stimulus	Health check probe sent to service
Environment	Normal operation; service running
Artifact	Health endpoint in `main.py`
Response	Returns `{"status": "healthy"}` with HTTP 200
Response Measure	Responds within 100 ms; 99.9% uptime; stateless restart

4.3.6 QAS-6: Security

Table 4.7: QAS-6: Security

Element	Description
Source	External user or malicious client
Stimulus	Sends non-image file or oversized payload (>10 MB)
Environment	Normal operation
Artifact	API route layer (`predict.py`)
Response	Rejected at route boundary with HTTP 400 before reaching pipeline
Response Measure	100% non-image uploads blocked; zero pipeline invocations with invalid types

4.3.7 QAS-7: Usability

Table 4.8: QAS-7: Usability

Element	Description
Source	Research user
Stimulus	Completes prediction and views result dashboard
Environment	Normal operation via web browser
Artifact	Frontend components: `ResultDisplay.jsx`, `Home.jsx`
Response	Dashboard displays species, health, confidence bars, and text summary
Response Measure	Result displayed within 500 ms of response; task completion >90% on first attempt

4.3.8 QAS-8: Interoperability

Table 4.9: QAS-8: Interoperability

Element	Description
Source	Frontend development team
Stimulus	Frontend updated; backend API contract must remain stable
Environment	Development and deployment environments
Artifact	`POST /api/predict` endpoint
Response	Frontend and backend communicate via multipart form with JSON metadata
Response Measure	Contract test pass rate 100%; response schema unchanged across releases

4.3.9 QAS-9: Deployability

Table 4.10: QAS-9: Deployability

Element	Description
Source	System administrator
Stimulus	Move deployment from local machine to cloud VM
Environment	Deployment transition
Artifact	Entire LeafGuard system
Response	System deployed and operational on cloud host
Response Measure	Completed within 30 min; rollback within 5 min; only env vars changed

4.3.10 QAS-10: Explainability

Table 4.11: QAS-10: Explainability

Element	Description
Source	Research user or lab supervisor
Stimulus	Requests understanding of prediction result
Environment	Normal operation; results displayed
Artifact	Result formatter and frontend dashboard
Response	Displays species, health, confidence scores, and natural-language summary
Response Measure	Summary comprehensible to non-technical researchers; confidence as percentages

5 Architecture Background

5.1 Design Rationale

LeafGuard adopts layered architecture at system scope to isolate user interaction, application orchestration, domain logic, and infrastructure concerns. This separation enables targeted evolution of frontend behavior and ML pipeline behavior without systemic refactoring.

Within the domain layer, a pipe-and-filter design was selected because ML inference naturally decomposes into explicit transformation and classification steps. Each filter owns one concern and has a narrow contract via a shared data payload. This design trades strict compile-time data contracts for runtime flexibility and rapid experimentation, aligning with the project's iterative research context.

The species-first then disease classification strategy reflects domain decomposition rather than purely algorithmic convenience. Species specialization allows distinct disease models and feature handling paths, but introduces artifact lifecycle complexity and risk of model drift. The architecture mitigates operational fragility via demo-mode fallback and explicit route-level validations.

The choice of React 18 with Vite for the frontend was driven by modern SPA capabilities and fast build times. FastAPI was selected for the backend due to native async support and tight integration with the Python ML ecosystem. The hybrid CNN+SVM approach leverages both visual features from leaf images and quantitative metadata capturing conditions not visible in images alone.

5.2 Analysis of Results

Table 5.1: Model Performance Summary

Model	Test Samples	Accuracy	F1 Score	Architecture
Species Classifier	1,055	96.68%	—	CNN (sigmoid)
AS Health Model	519	96.72%	0.9672	CNN + SVM (522-dim)
PC Health Model	653	98.77%	0.9877	CNN + SVM (521-dim)

The high accuracy across all models validates the hybrid CNN+SVM approach. The PC model's higher accuracy (98.77% vs. 96.72% for AS) may reflect more visually distinctive health markers in Prosopis Cineraria leaves or the larger training set (3,700 vs. 2,935 samples).

5.3 Assumptions

1. **Image Quality:** Input images are assumed to be reasonably clear photographs of individual leaves.
2. **Metadata Availability:** All 9 user-facing metadata fields are known at prediction time.
3. **Two-Species Scope:** Designed exclusively for AS and PC; extending to other species requires new model training.
4. **Stable Feature Schema:** The 10-feature tabular vector order must match SVM training schema.

5. **Single-User Load:** Designed for research use with low concurrency.
6. **Model Stationarity:** Deployed models remain valid for the current data distribution.

6 Architecture Overview and Styles

6.1 Architectural Approach

LeafGuard employs a **composition of architectural patterns** rather than a single monolithic style. This is consistent with Bass et al.'s observation that real systems typically combine multiple patterns [1]. The primary patterns are:

1. **Layered Architecture** at the system level for separation of concerns.
2. **Pipe-and-Filter** within the ML inference subsystem for modifiability and testability.
3. **Client-Server** between the React frontend and FastAPI backend.

6.2 Layered Architecture (System-Level)

6.2.1 Layer Definition

The system is decomposed into four layers with strict downward-only dependency rules:

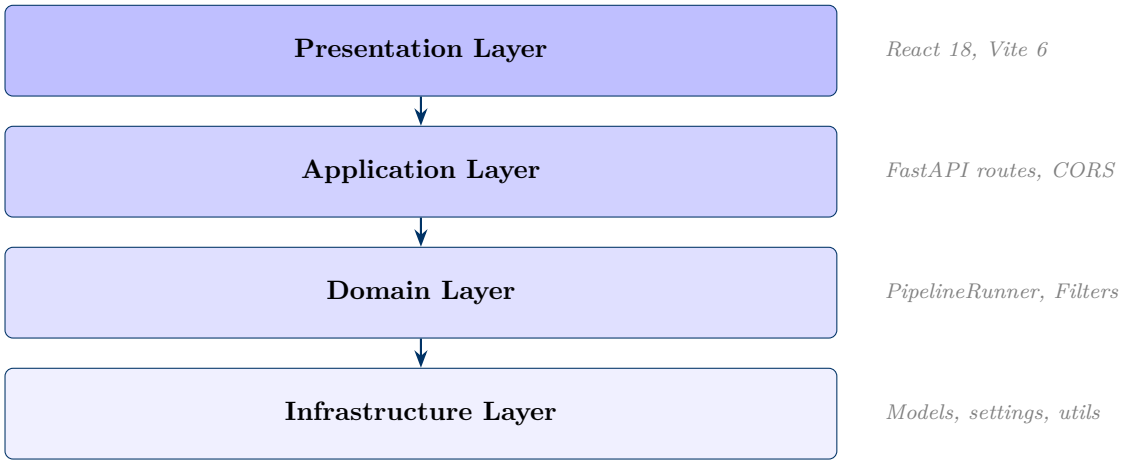

Figure 6.1: Layered Architecture of LeafGuard

6.2.2 Layer Responsibilities

Table 6.1: Layer Responsibilities and File Mapping

Layer	Key Files	Responsibilities
Presentation	`Home.jsx`, components, `api.js`	User interaction, image upload, metadata forms, result visualization
Application	`predict.py`, `main.py`	HTTP endpoints, request validation, CORS, error mapping
Domain	`prediction_service.py`, `pipeline/*.py`	Pipeline orchestration, classification, result formatting
Infrastructure	`settings.py`, `utils/*.py`, `models/`	Model paths, image conversion, metadata encoding

6.2.3 Rationale and Tradeoffs

Rationale: Separation of concerns for a research codebase that evolves rapidly. Frontend and ML pipeline can be modified independently.

Tradeoff: More files and cross-layer call overhead compared to a single-file scripting approach, but pays dividends in long-term maintainability.

6.3 Pipe-and-Filter (ML Inference Subsystem)

6.3.1 Pattern Description

The ML inference pipeline follows the Pipe-and-Filter architectural pattern (Shaw and Garlan [9]). Each filter is an independent processing stage that transforms a shared data dictionary.

6.3.2 Filter Chain

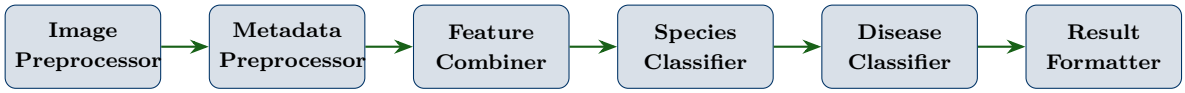

Figure 6.2: Pipe-and-Filter Pipeline: Six Inference Stages

6.3.3 Rationale and Tradeoffs

Rationale: High modifiability and testability. Each filter owns one concern. New stages added by subclassing `BaseFilter`.

Tradeoff: Dictionary-based coupling can hide schema contracts. Sequential execution prevents parallel processing of independent stages.

6.4 Client-Server Pattern

The React frontend communicates with the FastAPI backend over HTTP using a multipart form API. This separation enables:

- Independent deployment and scaling of frontend and backend.
- Frontend served as static files from any web host.
- Backend API consumable by alternative clients (CLI, mobile, scripts).

7 Architecture Views and Diagrams

This chapter presents architectural views following the C4 model and "Views and Beyond" approach (Clements et al. [2]).

7.1 View 1: System Context (C4 Level 1)

7.1.1 Primary Presentation

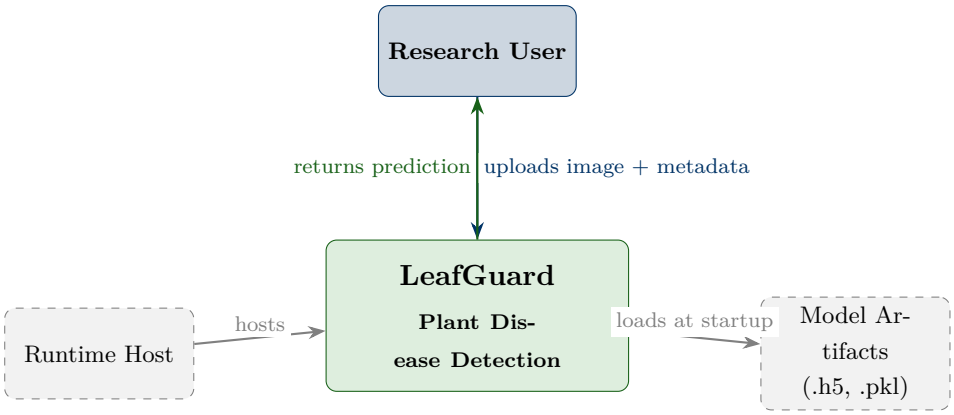

Figure 7.1: System Context Diagram (C4 Level 1)

7.1.2 Element Catalog

Table 7.1: System Context Element Catalog

Element	Type	Description
Research User	Person	Primary actor who uploads leaf images and reviews predictions
LeafGuard	System	Core platform providing species classification and health prediction
Model Artifacts	External	Pre-trained Keras CNN (.h5) and scikit-learn SVM (.pkl) files
Runtime Host	External	Local development machine or cloud VM

7.2 View 2: Container View (C4 Level 2)

7.2.1 Primary Presentation

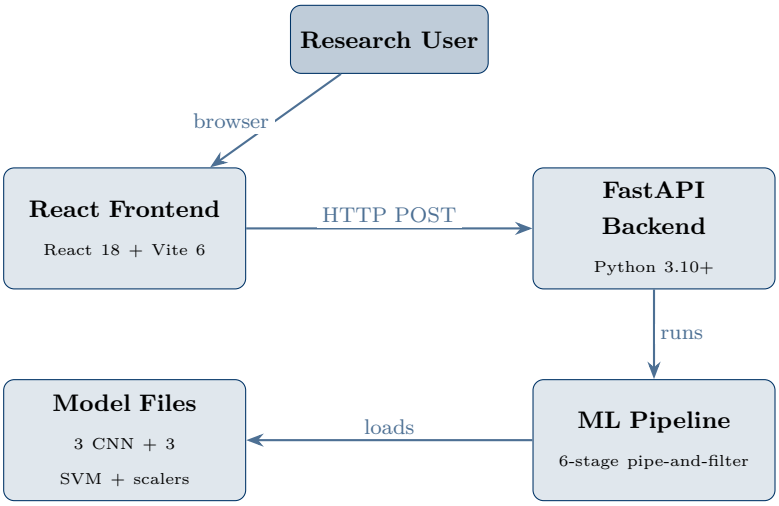

Figure 7.2: Container Diagram (C4 Level 2)

7.2.2 Element Catalog

Table 7.2: Container Element Catalog

Container	Technology	Responsibility
React Frontend	React 18, Vite 6	Image upload, metadata collection, result display
FastAPI Backend	FastAPI 0.115.6	Request validation, pipeline invocation, response formatting
ML Pipeline	TensorFlow, scikit-learn	6-stage inference: preprocess, classify, format
Model Files	.h5 (Keras), .pkl (joblib)	Pre-trained weights and scalers

7.2.3 Container Interfaces

Table 7.3: Container Interfaces

Interface	Method	Contract
`/api/predict`	POST	Multipart: `image` + `metadata` (JSON). Returns `{status, data}`
`/api/health`	GET	Returns `{status: "healthy"}`
`/api/pipeline-info`	GET	Returns `{stages: [...], num_stages: N}`

7.3 View 3: Backend Module View

7.3.1 Primary Presentation

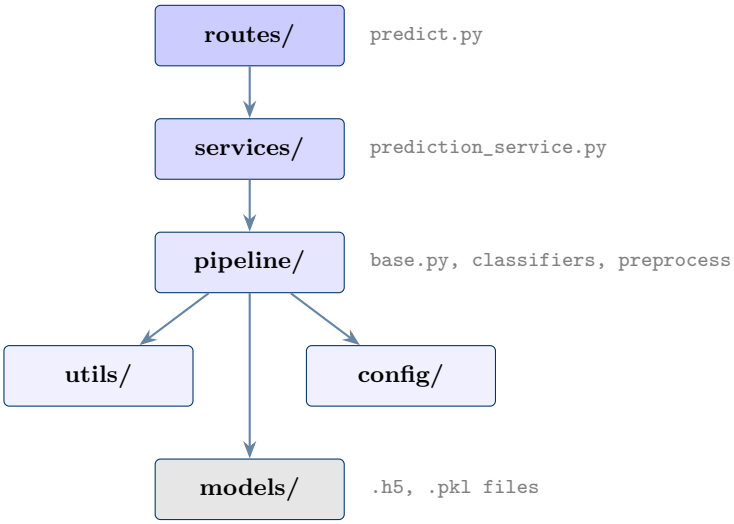

Figure 7.3: Backend Module View with Dependency Arrows

7.3.2 Dependency Rules

- Dependencies flow **downward only**; no upward dependencies exist.
- `routes/` depends on `services/` but never on `pipeline/` directly.
- `pipeline/` depends on `utils/` and `config/` for model paths and data utilities.
- `models/` contains only data artifacts—no code dependencies.

7.4 View 4: Pipe-and-Filter Runtime View (C&C)

7.4.1 Primary Presentation

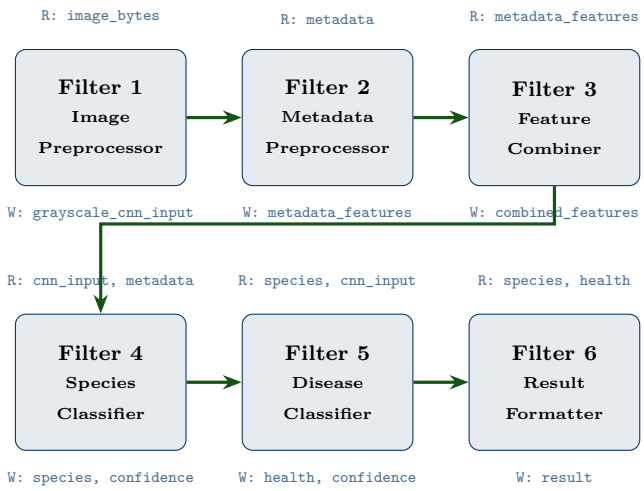

Figure 7.4: Pipe-and-Filter Runtime View with Read/Write Data Keys

7.4.2 Filter Behavior Catalog

Table 7.4: Pipeline Filter Behavior Catalog

Filter	Internal Behavior
ImagePreprocess	PIL Image from bytes → grayscale → 224×224 → normalize [0,1] → expand to (1,224,224,1)
MetadataPrepro	Validates 8 binary fields + Week range → derives Damage_missing → 10-feature float32 array
FeatureCombine	Copies metadata_features to combined_features (passthrough for future extension)
SpeciesClassifier	CNN predicts on grayscale: sigmoid <0.5=AS, ≥0.5=PC. Demo: heuristic from metadata
DiseaseClassifier	CNN→512d embedding + tabular features → SVM predict. AS: 522-dim, PC: 521-dim
ResultFormatter	Maps species code to full name → assembles {species, health, summary} JSON

7.5 View 5: Sequence Diagram – Predict Request

7.5.1 Successful Prediction Flow

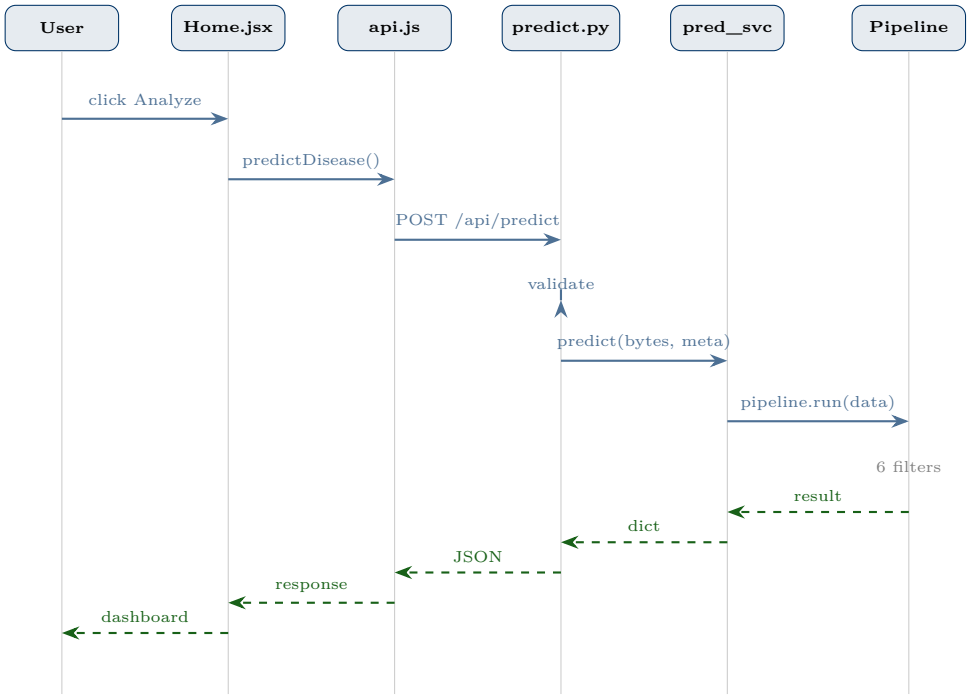

Figure 7.5: Sequence Diagram: Successful Prediction Request

7.5.2 Error Flow

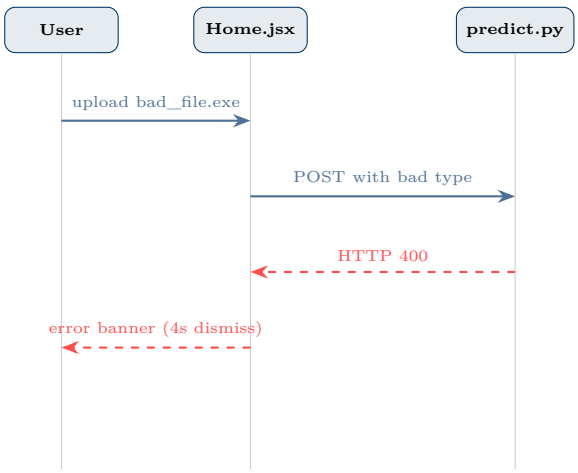

Figure 7.6: Sequence Diagram: Error Flow

7.6 View 6: Deployment View

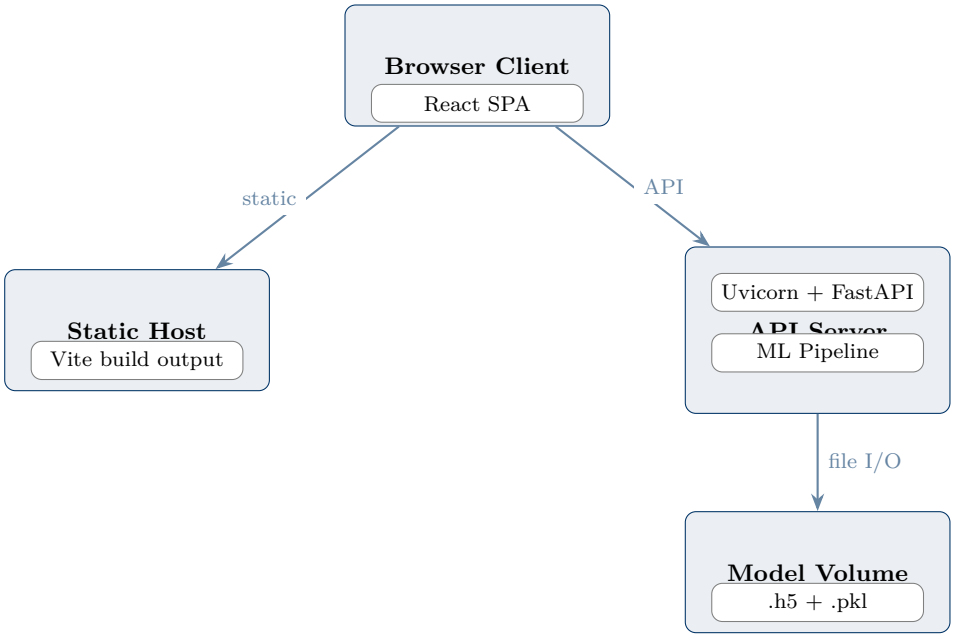

Figure 7.7: Deployment View

7.7 View 7: UI Interaction State Diagram

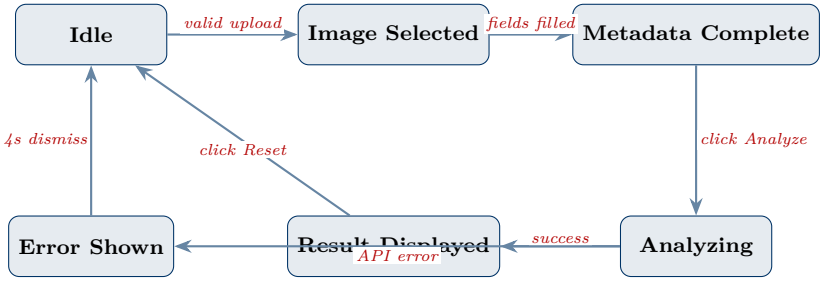

Figure 7.8: UI Interaction State Machine

7.8 Variability Guide

Table 7.5: Architecture Variability Points

Variability Point	Description	Mechanism
ML Model	CNN/SVM models replaceable with new versions	Config paths in `settings.py`
Species Support	New species can be added beyond AS/PC	New model files + routing
Pipeline Stages	New filters can be added	Subclass `BaseFilter`
Metadata Schema	Additional tabular features addable	Update `METADATA_FIELDS`
Deployment Mode	Switch demo/production	`DEMO_MODE` env var

8 Attribute-Driven Design (ADD) Walkthrough

This chapter documents the iterative decomposition of LeafGuard using ADD 2.0 [10], as recommended in Bass et al. [1].

8.1 Design Drivers

Prioritized from the utility tree (Table 4.1):

1. **Modifiability (H,H):** Pipeline stages must be independently replaceable.
2. **Performance (H,H):** End-to-end inference within 5 seconds.
3. **Testability (H,M):** Each component testable in isolation.
4. **Usability (H,M):** Results clearly presented with confidence indicators.
5. **Reliability (H,M):** Invalid inputs produce structured errors, never crashes.

8.2 Iteration 1: System Decomposition

Table 8.1: ADD Iteration 1: System Decomposition

Element	Description
Input Drivers	Modifiability, testability, quick deployment, usable UI
Design Decision	Layered system with frontend-backend separation
Chosen Pattern	Layered Architecture + Client-Server
Resulting Elements	React UI, FastAPI API, service layer, pipeline module, model assets
Rationale	Separation of presentation from domain logic enables independent evolution
Alternatives	Monolithic Python app with embedded templates → rejected: poor modifiability
Quality Impact	Modifiability (+), Testability (+), Deployability (+)

8.3 Iteration 2: Inference Pipeline Decomposition

Table 8.2: ADD Iteration 2: Pipeline Decomposition

Element	Description
Input Drivers	Modifiability, explainability, stage-level testing
Design Decision	Decompose ML inference into pipe-and-filter architecture
Chosen Pattern	Pipe-and-Filter with `BaseFilter` ABC
Resulting Stages	ImagePreprocessor → MetadataPreprocessor → FeatureCombiner → SpeciesClassifier → DiseaseClassifier → ResultFormatter
Rationale	ML inference naturally decomposes into transformation and classification steps
Alternatives	Single monolithic function → rejected: poor testability
Quality Impact	Modifiability (++), Testability (++), Explainability (+)

8.4 Iteration 3: Species/Disease Modeling Strategy

Table 8.3: ADD Iteration 3: Modeling Strategy

Element	Description
Input Drivers	Domain fidelity, confidence reporting, accuracy
Design Decision	Two-step classification: species first, then species-specific disease model
Resulting Decisions	Separate CNN+SVM per species; AS: 522-dim features; PC: 521-dim (drops Damage_missing); demo mode heuristic fallback
Rationale	Species specialization mirrors biological problem structure
Alternatives	Single unified model → rejected: different feature engineering per species
Quality Impact	Domain fidelity (++), Accuracy (++), Explainability (+)

8.5 Iteration 4: UX and Interaction Design

Table 8.4: ADD Iteration 4: UX Design

Element	Description
Input Drivers	Usability, result emphasis, error recovery
Design Decision	Split-panel layout: left input, right results dashboard
Chosen Mechanisms	Error banner with auto-dismiss; metadata hide-after-analyze; reset flow
Rationale	Results-first workflow for research demos
Alternatives	Full-page wizard → rejected: too many clicks for repeated analysis
Quality Impact	Usability (++), Learnability (+)

8.6 ADD Decomposition Tree

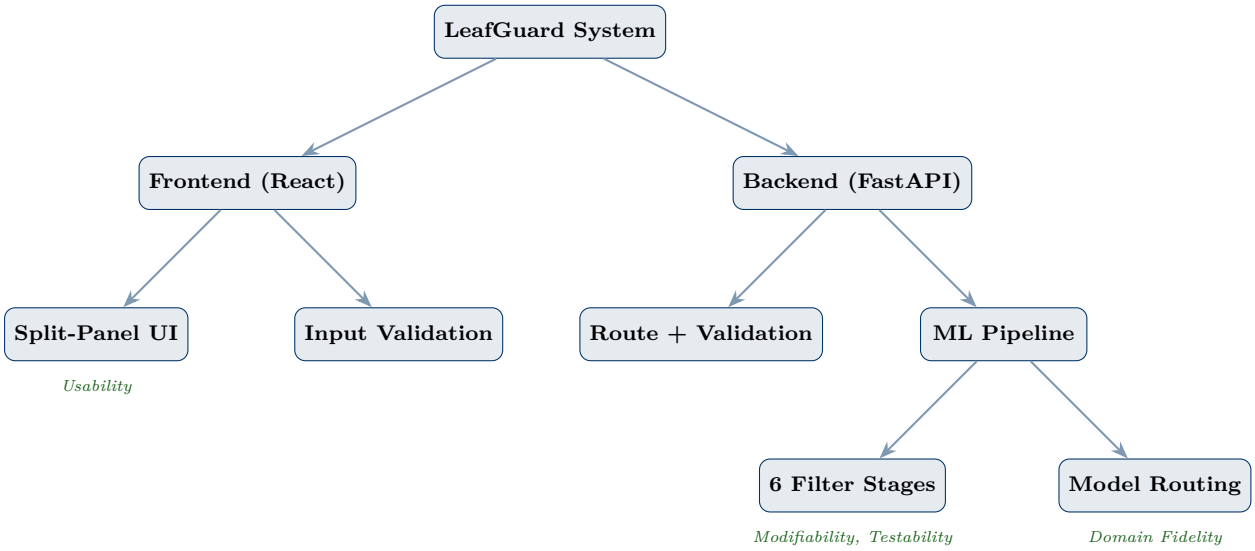

Figure 8.1: ADD Decomposition Tree with Quality Drivers

9 ML Subsystem Architecture Deep Dive

9.1 Overview

The ML subsystem implements a hybrid inference architecture combining CNNs [6] for image feature extraction with SVMs [3] for classification, augmented by tabular metadata features.

9.2 Inference Data Path

1. **Image Reception:** Raw bytes via HTTP multipart form upload.
2. **Image Preprocessing:** Bytes → PIL Image → grayscale → 224×224 → normalize [0,1] → (1,224,224,1).
3. **Metadata Validation:** 9 user fields validated; Damage_missing derived; 10-feature float32 vector.
4. **Species Classification:** CNN predicts species from grayscale image.
5. **Disease Classification:** Hybrid CNN+SVM uses image features and tabular metadata.
6. **Result Assembly:** Structured JSON with species, health, confidences, and summary.

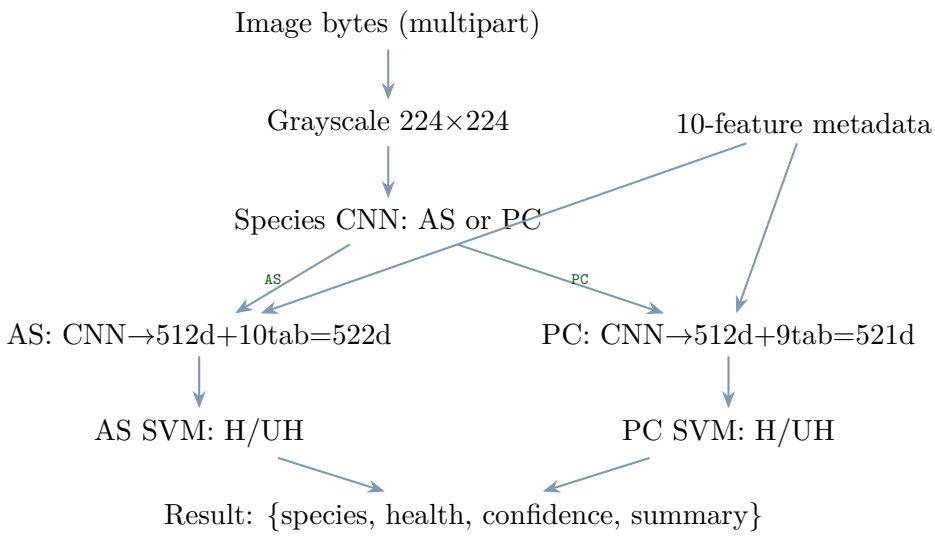

Figure 9.1: ML Inference Data Path with Species-Specific Routing

9.3 Model Catalog

9.3.1 Species Classifier (CNN)

Table 9.1: Species Classifier Details

Property	Value
Task	Binary species classification: AS vs PC
Input	Grayscale image, 224×224×1
Architecture	CNN (Keras .h5 model)
Output	Sigmoid: <0.5 = AS, ≥ 0.5 = PC
Test Set Size	1,055 images
Test Accuracy	**96.68%** (1,022 correct, 33 incorrect)

9.3.2 AS Disease Model (Hybrid CNN + SVM)

Table 9.2: AS Health Model Details

Property	Value
Task	Binary health classification: Healthy vs Unhealthy for AS leaves
Input	Grayscale 224×224 image + 10 tabular metadata features
Architecture	CNN → 512-dim features + 10 tabular → SVM
Feature Dimension	522 (512 CNN + 10 scaled tabular)
Total Samples	3,454 (2,935 train / 519 test)
Test Accuracy	**96.72%**
F1 Score	**0.9672**
CNN Scaler	Applied (StandardScaler on 512-dim embedding)

9.3.3 PC Disease Model (Hybrid CNN + SVM)

Table 9.3: PC Health Model Details

Property	Value
Task	Binary health classification: Healthy vs Unhealthy for PC leaves
Input	Grayscale 224×224 image + 9 tabular metadata features
Architecture	CNN → 512-dim features + 9 tabular → SVM
Feature Dimension	521 (512 CNN + 9 tabular; Damage_missing dropped)
Total Samples	4,353 (3,700 train / 653 test)
Test Accuracy	**98.77%**
F1 Score	**0.9877**
CNN Scaler	**Not applied** (raw embedding)

9.4 Model Routing Architecture

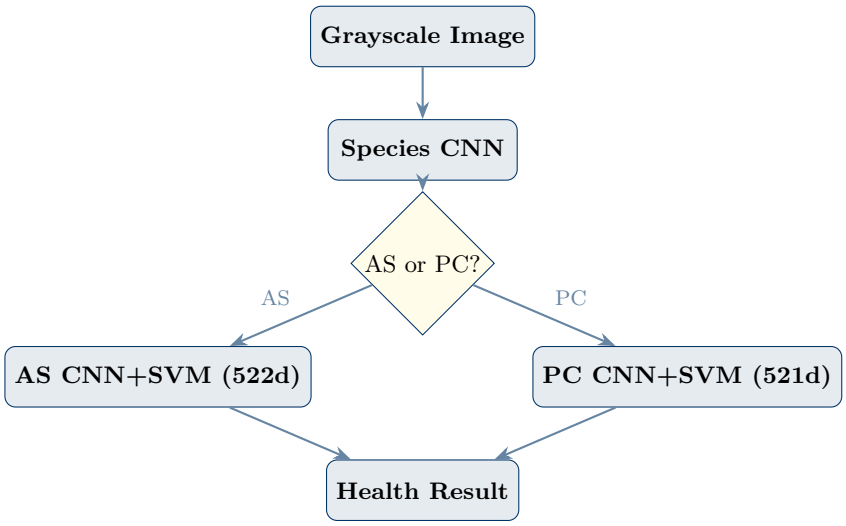

Figure 9.2: Species-Based Model Routing

9.5 Demo Mode Heuristic

When model files are unavailable (`DEMO_MODE=true`), rule-based heuristics are used:

Species: Computes AS score from Week, Drought, and Heat_Drought metadata. Score ≥ 0.4 $\rightarrow$ AS; otherwise $\rightarrow$ PC. Confidence: 0.65–0.90.

Disease: Computes unhealthy score from Spots, Decolourization, Damage, Heat, Drought. Score ≥ 0.3 $\rightarrow$ Unhealthy; otherwise $\rightarrow$ Healthy. Confidence: 0.60–0.92.

10 Tradeoffs, Tactics, and Sensitivity Analysis

10.1 Architectural Tactics

Architectural tactics are design decisions that directly influence quality attribute responses [1].

Table 10.1: Architectural Tactics

Decision/Tactic	Primary QA	Cost/Risk	Why Chosen
Preload models at startup	Performance	Startup heavier	Inference latency improvement outweighs startup cost
Six-stage filter pipeline	Modifiability	Schema discipline	Independent stage evolution and testing
Route-level input guards	Security	Some edge cases rejected	Early rejection prevents deep pipeline failures
Demo mode fallback	Availability	Non-real risk	Development without full model bundle
Species-first classification	Domain fidelity	Extra models	Mirrors biological problem structure
Auto-dismiss error banner	Usability	May miss brief errors	Keeps UI clean during repeated experiments
Stateless API design	Deployability	Context per-request	Enables scaling, simple restart/rollback
CORS origin whitelist	Security	Must update for new URLs	Prevents unauthorized cross-origin access

10.2 ATAM-Lite: Sensitivity Points

Sensitivity points are architectural decisions where a small change significantly affects a quality attribute [5]:

1. **S1 — Pipeline Filter Order:** Reordering filters breaks data dependencies. Current order determined by data flow.
2. **S2 — Model Loading Strategy:** Changing from startup to lazy loading impacts first-request latency and memory patterns.
3. **S3 — Feature Dimension Mismatch:** CNN embedding dimension must be 512 for SVM compatibility. AS expects 522 features; PC expects 521.
4. **S4 — Tabular Feature Order:** The 10-element metadata vector must match exact column order from SVM training (`METADATA_FIELDS`).
5. **S5 — Confidence Threshold:** Species classifier uses 0.5 sigmoid threshold. Adjusting this significantly changes behavior.

11 Verification, Testing, and Evolution Plan

11.1 Test Strategy

The testing strategy follows a pyramid approach:

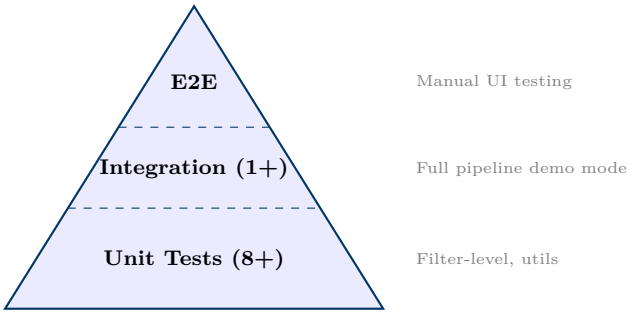

Figure 11.1: Test Pyramid for LeafGuard

11.2 Current Test Coverage

Table 11.1: Test Inventory

Test Name	Type	Verification	Status
test_validate_encode_meta	Unit	10 fields encoded; Damage_missing=1	Pass
test_validate_metadata_rej	Unit	Missing "Spots" raises ValueError	Pass
test_validate_metadata_rej	Unit	"maybe" for Heat raises ValueError	Pass
test_encode_to_array_shap	Unit	Shape (10,) and dtype float32	Pass
test_image_preprocessor_o	Unit	Shape (1,224,224,1); range [0,1]	Pass
test_metadata_preprocesso	Unit	Shape (10,); Damage_missing at idx 8	Pass
test_feature_combiner_pas	Unit	Output equals input; shape (10,)	Pass
test_full_pipeline_demo_m	Integ.	Full pipeline produces valid result	Pass

11.3 ADD Validation Evidence

Table 11.2: ADD Iteration Validation Evidence

Iteration	Decision	Validation Evidence
1	Frontend/backend separation	Independent builds: `npm run build` + `uvicorn`
2	Pipe-and-filter pipeline	7 unit tests verify each filter independently
3	Species-specific models	Accuracy validated: 96–99% on held-out test sets
4	UX split-panel design	Manual UI testing; error banner auto-dismiss works

11.4 Architectural Decision Records (ADR Summary)

Table 11.3: ADR Summary

ADR	Decision	Alternatives	Rationale	QA Impact	Tradeoff
1	Layered architecture	Monolithic; microservices	Separation of concerns	Modif.+, Test.+	Complexity
2	Pipe-and-filter ML	Monolithic function	Natural ML decomposition	Modif.++, Test.++	Schema discipline
3	Species-first classify	Unified model	Domain fidelity	Accuracy+	More artifacts
4	Hybrid CNN+SVM	CNN-only; ML-only	Visual + tabular features	Accuracy++	Training complexity
5	React + FastAPI	Django; Flask+Jinja	Modern SPA; Python ML	Usability+	Two tech stacks
6	Demo mode fallback	Error on missing models	Dev without models	Availability+	Misleading if prod
7	Dict-based pipe payload	Typed dataclass	Flexible for research	Flexibility+	No type safety
8	Auto-dismiss error	Persistent error; modal	Clean UX	Usability+	May miss errors

11.5 Evolution Roadmap

Table 11.4: Architecture Evolution Roadmap

Phase	Description	Timeline
Phase 1	Schema validation for pipe payload (Pydantic models)	Near-term
Phase 2	Model registry with version tracking	Near-term
Phase 3	Structured logging and tracing (OpenTelemetry)	Mid-term
Phase 4	CI/CD pipeline with automated testing	Mid-term
Phase 5	Support additional species beyond AS and PC	Long-term
Phase 6	Containerize with Docker	Mid-term
Phase 7	Authentication and rate limiting	Long-term

12 Glossary of Terms

Table 12.1: Glossary of Technical Terms

Term	Definition
ADD	Attribute-Driven Design. Iterative architectural design method using quality attribute scenarios as drivers [10].
API	Application Programming Interface. HTTP-based interface exposed by FastAPI backend.
AS	Acacia Senegal. One of two plant species supported by LeafGuard.
ATAM	Architecture Tradeoff Analysis Method [5].
BaseFilter	Abstract base class defining `process(data)` contract for pipeline filters.
C4 Model	Hierarchical architecture visualization: Context, Container, Component, Code.
CNN	Convolutional Neural Network for image feature extraction [6].
CORS	Cross-Origin Resource Sharing. HTTP mechanism allowing frontend-backend communication.
Demo Mode	Configuration mode using heuristic rules instead of ML models.
FastAPI	High-performance Python web framework [8].
Glassmorphism	UI design style using frosted-glass effects.
H5	HDF5 format for storing Keras model weights.
Hybrid Model	CNN+SVM approach combining image and tabular features.
Keras	High-level neural network API in TensorFlow [4].
PC	Prosopis Cineraria. One of two plant species supported.
Pickle (.pkl)	Python serialization format for SVM models and scalers.
Pipe-and-Filter	Architectural pattern with sequential processing stages [9].
Quality Attribute	Measurable system property (performance, modifiability, etc.) [1].
SAIP	Software Architecture in Practice (Bass, Clements, Kazman) [1].
SVM	Support Vector Machine for health classification [3].
Utility Tree	ATAM artifact prioritizing quality attribute scenarios.
Uvicorn	High-performance ASGI server for FastAPI.
Vite	Frontend build tool for the React application [7].

13 Other Information

13.1 Technology Stack Summary

Table 13.1: Complete Technology Stack

Layer	Technology	Version	Purpose
Frontend	React	18.3.1	UI component framework
Frontend	Vite	6.0.0	Build tool and dev server
Backend	FastAPI	0.115.6	REST API framework
Backend	Uvicorn	0.34.0	ASGI server
Backend	Python	3.10+	Runtime environment
ML	TensorFlow	$\geq$2.16	CNN inference
ML	scikit-learn	1.6.1	SVM classifier and scalers
ML	Pillow	11.1.0	Image processing
Testing	pytest	8.3.4	Test framework

13.2 Repository Structure

```
leafguard/
  backend/
    app/
      main.py                     # FastAPI entry point
      config/settings.py          # Model paths, metadata schema
      routes/predict.py           # API endpoints
      services/prediction_service.py  # Pipeline instance
      pipeline/
        base.py                   # BaseFilter ABC
        pipeline_runner.py        # Sequential orchestrator
        preprocess.py             # Image preprocessing
        metadata_processor.py     # Metadata validation
        feature_combiner.py       # Feature combination
        species_classifier.py     # Species CNN
        disease_classifier.py     # Disease CNN+SVM
        result_formatter.py       # Result assembly
      utils/
        image_utils.py            # PIL image conversion
        metadata_utils.py         # Metadata encoding
      models/                     # .h5 + .pkl artifacts
    tests/test_pipeline.py        # pytest suite
  frontend/
    src/
      pages/Home.jsx              # Main page
      components/                 # UI components
      services/api.js             # API client
  requirements.txt
```

Listing 13.1: Repository Structure

13.3 Build and Run Instructions

Backend:

```
cd leafguard
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
uvicorn backend.app.main:app --reload
```

Listing 13.2: Backend Setup

Frontend:

```
cd leafguard/frontend
npm install
npm run dev     # Development
npm run build   # Production build
```

Listing 13.3: Frontend Setup

References

[1] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. 4th ed. Pearson Education, 2024. ISBN: 978-0136885979.

[2] Paul Clements et al. *Documenting Software Architectures: Views and Beyond*. 2nd ed. Addison-Wesley Professional, 2010. ISBN: 978-0321552686.

[3] Corinna Cortes and Vladimir Vapnik. "Support-Vector Networks". In: *Machine Learning* 20.3 (1995), pp. 273–297.

[4] Google Brain Team. *TensorFlow: An End-to-End Open Source Machine Learning Platform*. https://www.tensorflow.org/. Version 2.16+. 2024.

[5] Rick Kazman, Mark Klein, and Paul Clements. "ATAM: Method for Architecture Evaluation". In: *Technical Report CMU/SEI-2000-TR-004*. Software Engineering Institute, Carnegie Mellon University, 2000.

[6] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. "Deep Learning". In: *Nature* 521.7553 (2015), pp. 436–444.

[7] Meta Platforms, Inc. *React: A JavaScript Library for Building User Interfaces*. https://react.dev/. Version 18.3. 2024.

[8] Sebastián Ramírez. *FastAPI: Modern, Fast Web Framework for Building APIs with Python*. https://fastapi.tiangolo.com/. Version 0.115.6. 2024.

[9] Mary Shaw and David Garlan. "Software Architecture: Perspectives on an Emerging Discipline". In: *Prentice Hall* (1996).

[10] Rob Wojcik et al. *Attribute-Driven Design (ADD), Version 2.0*. Tech. rep. CMU/SEI-2006-TR-023. Software Engineering Institute, Carnegie Mellon University, 2006.